

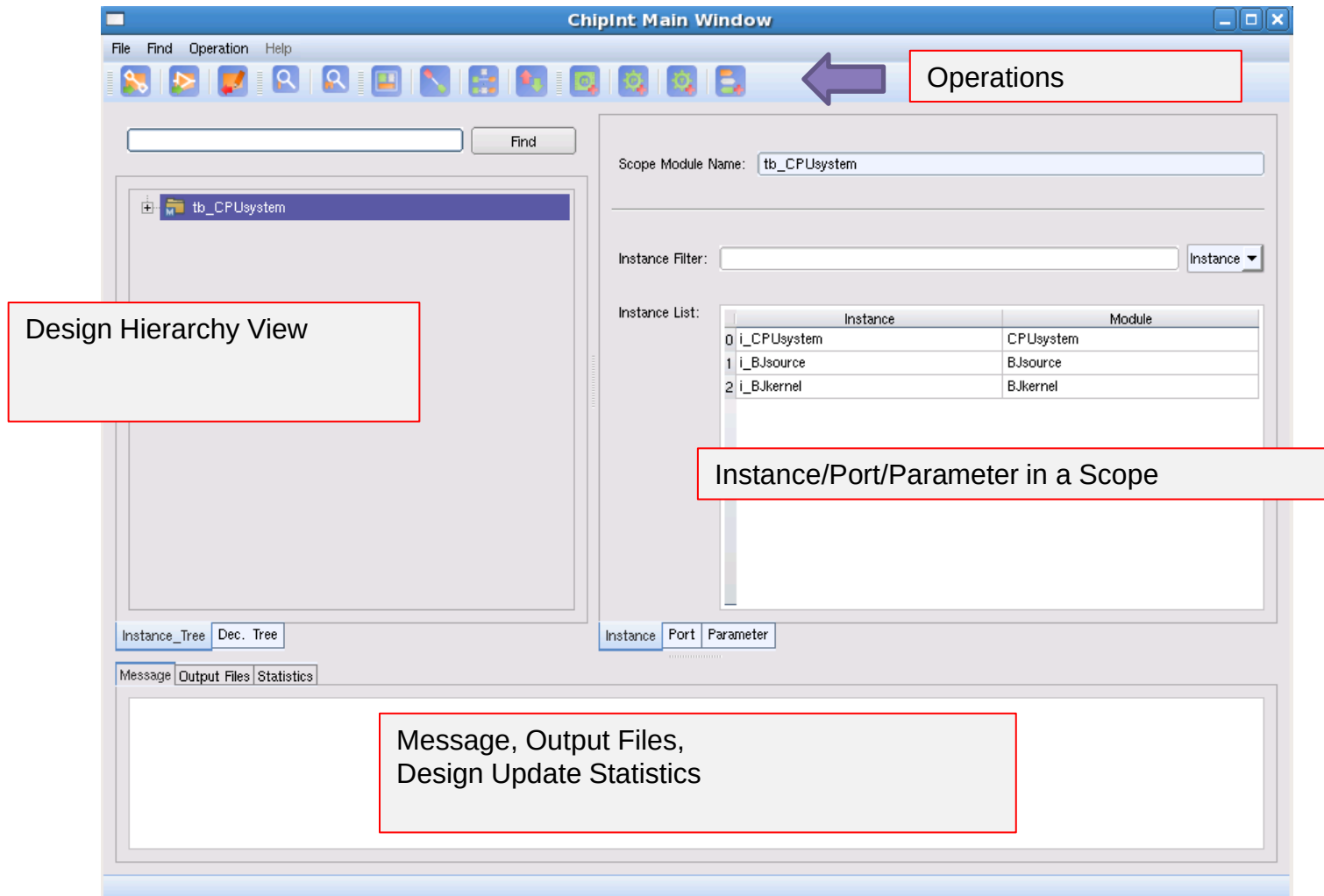
Outlines

- Overview
- Main Features

Features

- Robust and rapid SoC integration tool with underlying features
 - Integration operations
 - Module Instantiation
 - Port-to-Port Connection
 - Port-to-Hierarchy
 - Instance Insertion
 - Creation operations
 - Module Creation
 - Port Creation
 - Parameter Creation
 - Connection checking
 - IP reuse from coherent setting files
- Complete SoC integration flow
 - Build system hierarchy
 - IP instantiation
 - IP connection
 - IP reuse

ChipInt Main Window



Outlines

- Overview
- Main Features
 - Integration operations
 - **Module Instantiation**
 - Port-to-Port Connection
 - Port-to-Hierarchy
 - Instance Insertion
 - Creation operations
 - Module Creation
 - Port Creation
 - Parameter Creation
 - Connection checking

Module Instantiation - GUI

New Instance

Target Scope:

Source Module Name:

New Instance Name:

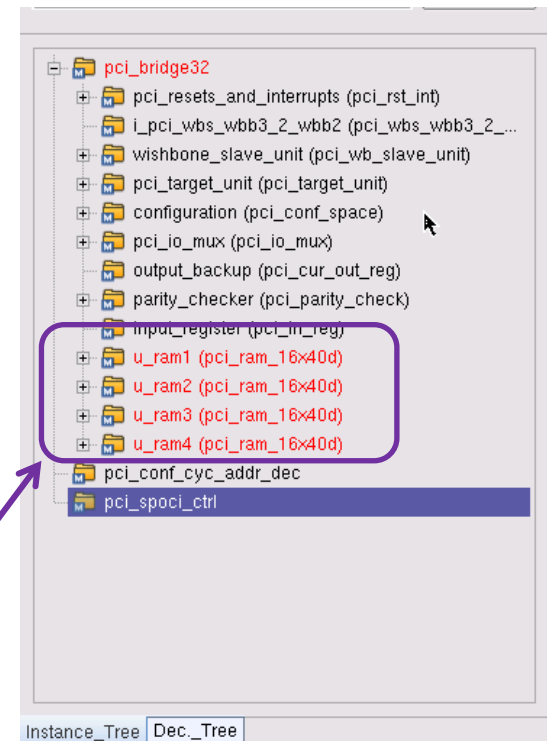
Assign parameter value/type:

	Param. List	Constant Value
0	addr_width	8

Type Param. List	Type
------------------	------

New instance list:

	Target Scope	Module Name	Instance Name	Parameter Assignment
0	pci_bridge32	pci_ram_16x40d	u_ram1	
1	pci_bridge32	pci_ram_16x40d	u_ram2	
2	pci_bridge32	pci_ram_16x40d	u_ram3	(1) addr_width : 2
3	pci_bridge32	pci ram 16x40d	u ram4	(1) addr_width : 8



Module Instantiation – Import by File

Target Scope:

Source Module Name:

New Instance Name:

Assign parameter value/type:

New instance list:

	Target Scope	Module Name	Instance Name	Parameter Assignment
0	top	p1_module	test_p1_instance_0	
1	top	p1_module	test_p1_instance_1	
2	chip	p1_module	test_p1_instance	
3	p1_top_h1	p1_module	test_p1_instance	

top, p1_module, test_p1_instance_0
top, p1_module, test_p1_instance_1
chip, p1_module, test_p1_instance
p1_top_h1, p1_module, test_p1_instance

Outlines

- Overview
- Main Features
 - Integration operations
 - Module Instantiation
 - Port-to-Port Connection
 - Port-to-Hierarchy
 - Instance Insertion
 - Creation operations
 - Module Creation
 - Port Creation
 - Parameter Creation
 - Connection checking

Port-to-Port Connection

Select operation modes

- Port-to-Port
- Port-to-Constant
- Port-to-Wild
- Interface-to-Interface
- Interface-to-Wild

The screenshot shows the 'Connect Port' dialog box with two main panels: 'Source Instance Setting' and 'Target Instance Setting'.

Source Instance Setting:

- Source Instance: `top.u chip.p1 inst`
- Name Rule: `prefix` (dropdown), Name Context: (empty)
- Source Instance Port List table:

	Name	Dir.	Packed Dim.	Target Connection
0	in_bottomup	input		
1	out_bottomup	output		
2	in_topdown	input		
3	out_topdown	output		
4	in_mix	input		<code>in_mix => top.u chip.p2 inst.in_mix</code>
5	out_mix	output		<code>out_mix => top.u chip.p2 inst.out_mix</code>
6	in_const	input		

Assignment in Index/Slice: (empty)

Target Instance Setting:

- Operation modes: `Port-to-Port`, `Port-to-Constant`, `Port-to-Wild`, `Interface-to-Interface` (selected), `Interface-to-Wild`
- Target Instance: `top.u chip.p2 inst` (with 'browse' button)
- Target Instance Port List table:

	Name	Dir.	Packed Dim.
0	in_mix	input	
1	out_mix	output	

Assignment in Index/Slice: (empty)

Buttons at the bottom: Apply, Ok, Cancel.

Port-to-Port

You can do port-to-port assignment one by one in this mode

1) select the port

2) select the target instance

3) select the port in the target instance

4) add to the target connection to the port in source

Source Instance Setting

Source Instance: top.u chip.p1 inst

Name Rule: prefix Name Context:

Source Instance Port List

	Packed Dim.	Target Connection
1 in_topdown	input	
2 out_topdown	output	
3 in_mix	input	in_mix => top.u chip.p2 inst.in_mix
4 out_mix	output	out_mix => top.u chip.p2 inst.out_mix
5 in_const	input	

Target Instance Setting

Port-to-Port Port-to-Constant Port-to-Wild

Target Instance: top.u chip.p2 inst

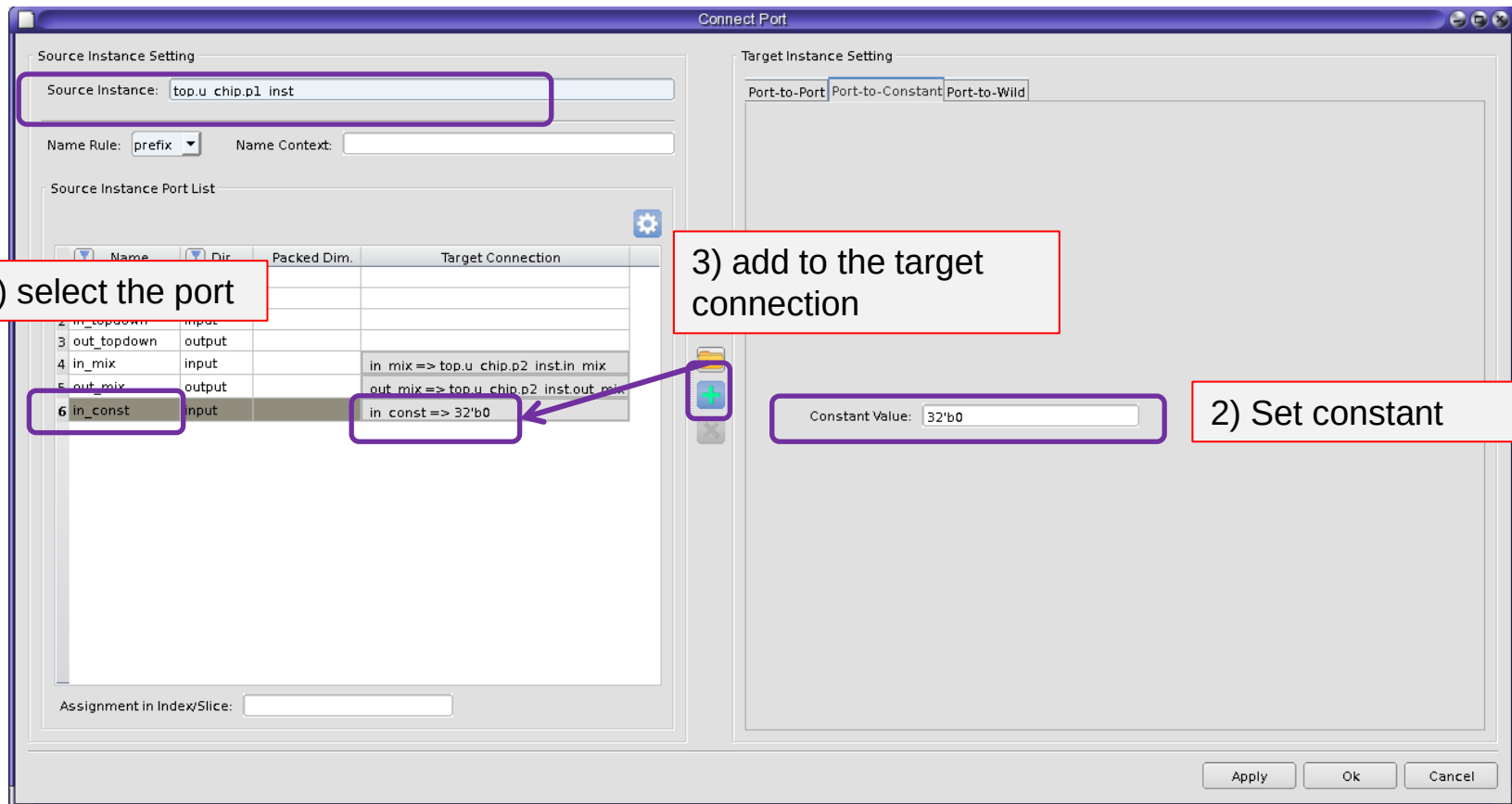
Target Instance Port List

	Name	Dir	Packed Dim.
0	in_mix	input	
1	out_mix	output	

Assignment in Index/Slice:

Port-to-Constant

You can do port-to-constant assignment one by one in this mode.



Port-to-Wild

You can do multiple port-to-port assignment at once with specified naming rule

Source Instance Setting

Source Instance: `top.u_chip.u_master`

Name Rule: `prefix` Name Context:

Source Instance Port List

	Name	Dir.	Packed Dim.	Target Connection
0	<code>bus_clk</code>	output		<code>bus_clk => top.u_chip.u_ip1.bus_clk</code> <code>bus_clk => top.u_chip.u_ip2.bus_clk</code> <code>bus_clk => top.u_chip.u_ip3.bus_clk</code>
1	<code>bus_rw</code>	output		<code>bus_rw => top.u_chip.u_ip1.bus_rw</code> <code>bus_rw => top.u_chip.u_ip2.bus_rw</code> <code>bus_rw => top.u_chip.u_ip3.bus_rw</code>
2	<code>bus_sel_ip1</code>	output		<code>bus_sel ip1 => top.u_chip.u_ip1.bus_sel ip1</code>
3	<code>bus_sel_ip2</code>	output		<code>bus_sel ip2 => top.u_chip.u_ip2.bus_sel ip2</code>
4	<code>bus_sel_ip3</code>	output		<code>bus_sel ip3 => top.u_chip.u_ip3.bus_sel ip3</code>
5	<code>bus_addr</code>	output	[11:0]	<code>bus_addr => top.u_chip.u_ip1.bus_addr</code> <code>bus_addr => top.u_chip.u_ip2.bus_addr</code> <code>bus_addr => top.u_chip.u_ip3.bus_addr</code>
6	<code>bus_rdy_ip1</code>	input		<code>bus_rdy ip1 => top.u_chip.u_ip1.bus_rdy ip1</code>
7	<code>bus_rdy_ip2</code>	input		<code>bus_rdy ip2 => top.u_chip.u_ip2.bus_rdy ip2</code>
8	<code>bus_rdy_ip3</code>	input		<code>bus_rdy ip3 => top.u_chip.u_ip3.bus_rdy ip3</code>
9	<code>bus_wdata</code>	output	[31:0]	<code>bus_wdata => top.u_chip.u_ip1.bus_wdata</code> <code>bus_wdata => top.u_chip.u_ip2.bus_wdata</code> <code>bus_wdata => top.u_chip.u_ip3.bus_wdata</code>
10	<code>bus_rdata_ip1</code>	input	[31:0]	<code>bus_rdata ip1 => top.u_chip.u_ip1.bus_rdata ip1</code>
11	<code>bus_rdata_ip2</code>	input	[31:0]	<code>bus_rdata ip2 => top.u_chip.u_ip2.bus_rdata ip2</code>
12	<code>bus_rdata_ip3</code>	input	[31:0]	<code>bus_rdata ip3 => top.u_chip.u_ip3.bus_rdata ip3</code>

Assignment in Index/Slice:

Target Instance Setting

Port-to-Port | Port-to-Constant | Port-to-Wild

Instance Setting

☒ Wild Card

Target Instance: `*ip*` Browse

Instance List

- `top.u_chip.u_ip1`
- `top.u_chip.u_ip2`
- `top.u_chip.u_ip3`

Mapping Name Setting

Mapping Name: `*bus_rdata*`

Mapping Name List

- `bus_clk`
- `bus_rw`
- `*bus_sel*`
- `bus_addr`
- `*bus_rdy*`
- `bus_wdata`
- `*bus_rdata*`

Annotations:

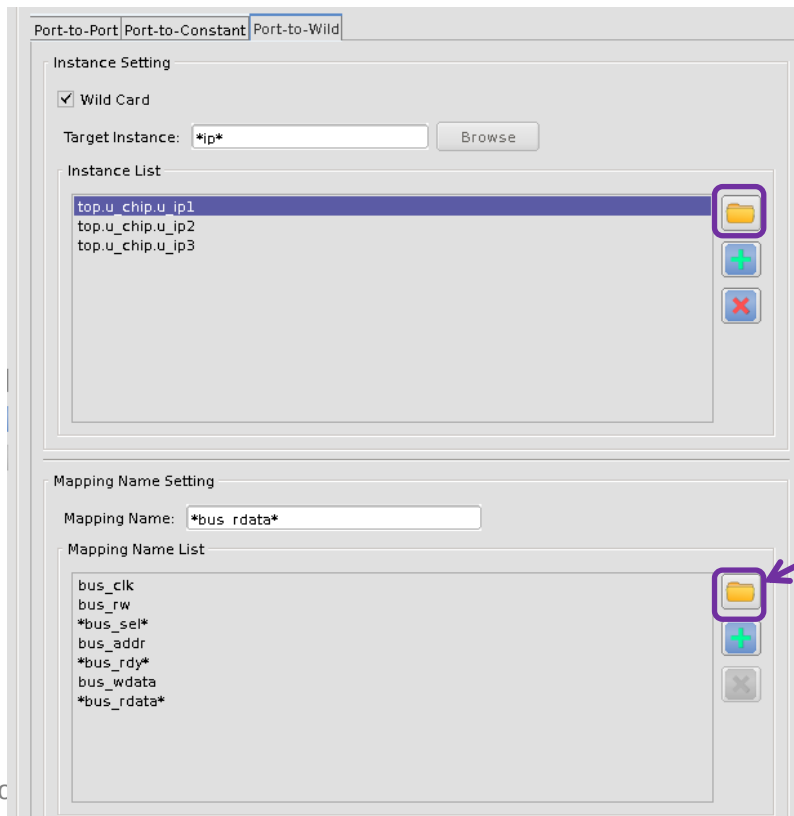
- 1) select wild card mode to get target instance list
- 2) Set target instance with wild card
- 3) Add multiple target instances with one button
- 4) Edit mapping name list
- 5) Add name list
- 6) Assign multiple connections at once

Port-to-Wild

- Wildcard name mapping
 - Only identical names are matched
 - HCLK (HCLK <-> HCLK)
 - HREADY*
 - HREADY_S0 <-> HREADY_S0
 - HREADY_S1 <-> HREADY_S1
 -
 - *HREADY
 - S0_HREADY <-> S0_HREADY
 - S1_HREADY <-> S1_HREADY
 -
 - *HREADY*
 - S0_HREADY_0 <-> S0_HREADY_0
 - S1_HREADY_1 <-> S1_HREADY_1
 -

Port-to-Wild

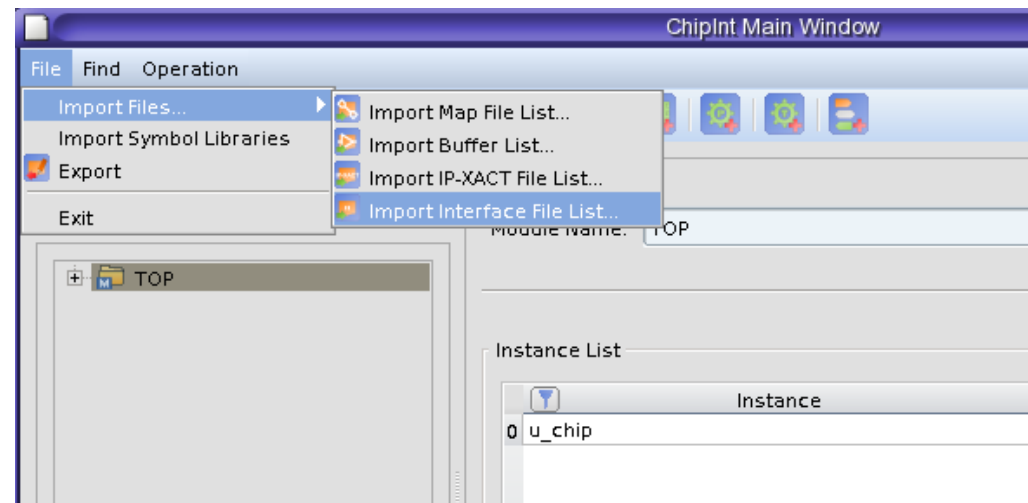
- You can import files to set instance list and mapping name list
 - The file can be reused for certain bus



bus_clk
bus_rw
bus_sel
bus_addr
bus_rdy
bus_wdata
bus_rdata

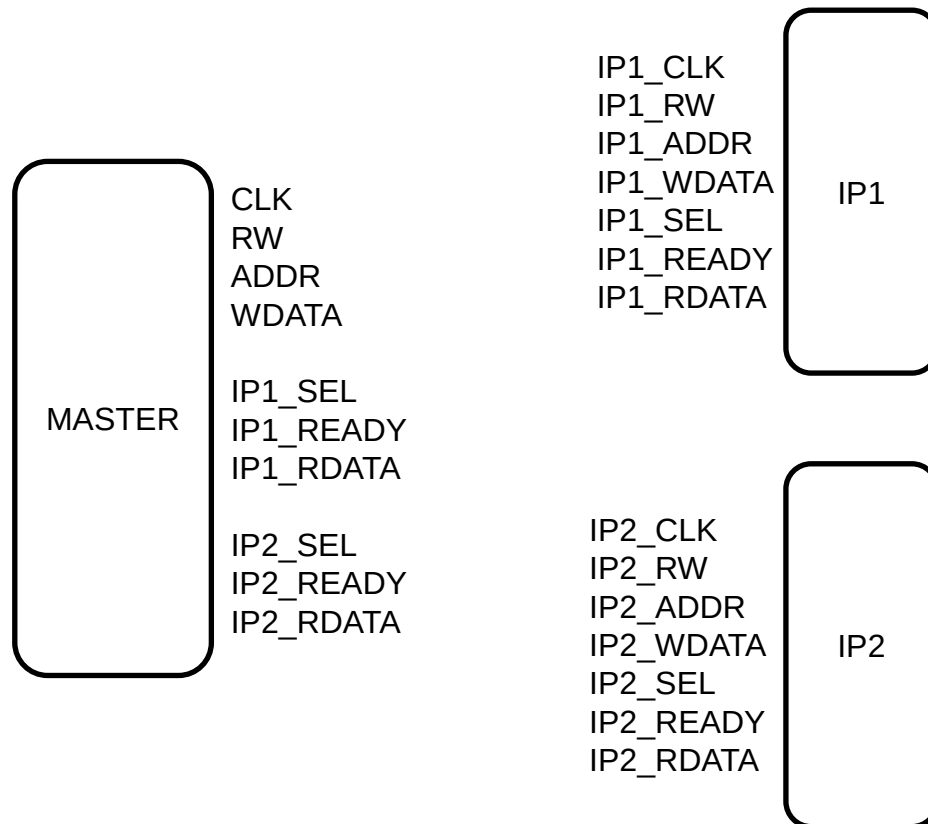
Interface-to-Interface

- Import Bus Interface Configuration File List
 - The list of file paths
- Example
 - ./testcase/ip1.int
 - ./testcase/ip2.int
 - ./testcase/master.int



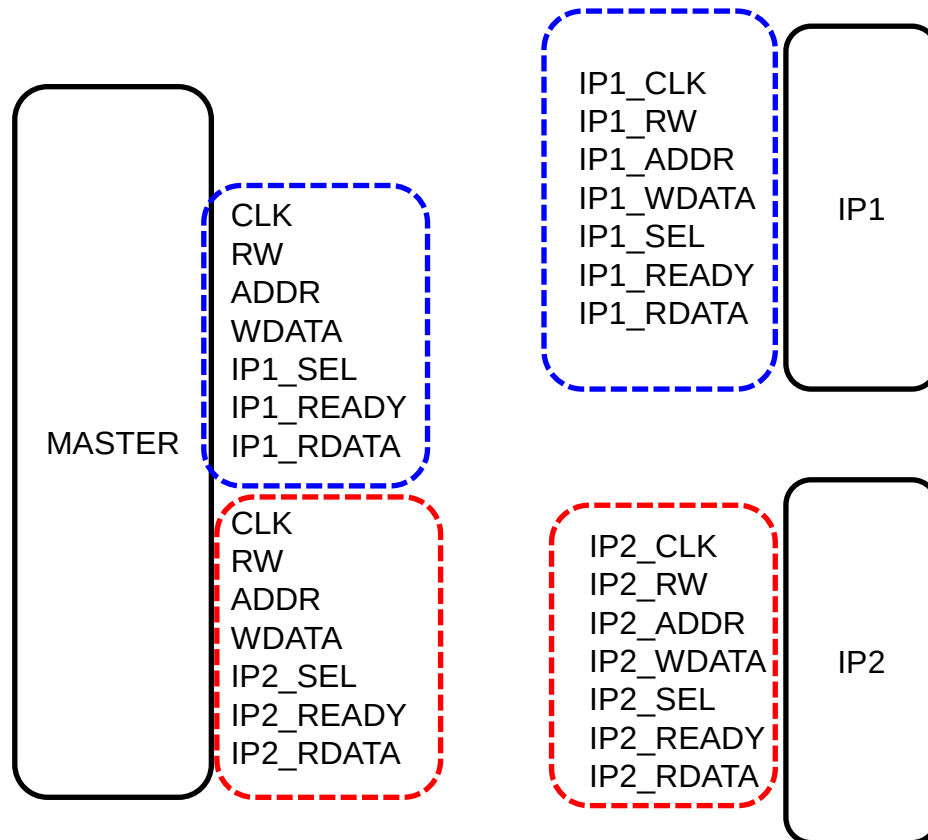
Bus Interface Configuration File

- Take a bus connection below for example:



Bus Interface Configuration File

- Group interface



Interface Configuration File Format

```
#####
#### Stage 1: Module name   ###
#####

<MODULE_NAME>
MASTER

#####
#### Stage 2: Interface list #####
#####

## For IP1
## "Interface name" -> "Bus type"
<BUS_INTERFACE>
IP1_INT BUS

## "Physical port name" -> "Logical port name"
<P2L_MAP>
CLK      CLK
RW       RW
ADDR     ADDR
WDATA    WDATA
IP1_SEL  SEL
IP1_RDY  RDY
IP1_RDATA RDATA

## For IP2
## "Interface name" -> "Bus type"
<BUS_INTERFACE>
IP2_INT BUS

## "Physical port name" -> "Logical port name"
<P2L_MAP>
CLK      CLK
RW       RW
ADDR     ADDR
WDATA    WDATA
IP2_SEL  SEL
IP2_RDY  RDY
IP2_RDATA RDATA

~
~
~
~
~
~
./master_type1.int
```

```
#####
#### Stage 1: Module name   ###
#####

<MODULE_NAME>
IP2

#####
#### Stage 2: Interface list #####
#####

## "Interface name" -> "Bus type"
<BUS_INTERFACE>
BUS_INT BUS

## "Physical port name" -> "Logical port name"
<P2L_MAP>
IP2_CLK  CLK
IP2_RW   RW
IP2_ADDR ADDR
IP2_WDATA WDATA
IP2_SEL  SEL
IP2_RDY  RDY

./ip2_type1.int

#####
#### Stage 1: Module name   ###
#####

<MODULE_NAME>
IP1

#####
#### Stage 2: Interface list #####
#####

## "Interface name" -> "Bus type"
<BUS_INTERFACE>
BUS_INT BUS

## "Physical port name" -> "Logical port name"
<P2L_MAP>
IP1_CLK  CLK
IP1_RW   RW
IP1_ADDR ADDR
IP1_WDATA WDATA
IP1_SEL  SEL
IP1_RDY  RDY

ip1_type1.int
```

Interface-to-Interface Flow

Source Instance Setting

Source Instance:

Name Rule: Name Context:

Source Instance Port List

Name	Dir.	Packed Dim.	Bus Interface	Logical Definition	st Contr
0 CLK	input		(1) IP1_INT [BUS] (2) IP2_INT [BUS]	(1) CLK [BUS] (2) CLK [BUS]	
1 RW	output		(1) IP1_INT [BUS] (2) IP2_INT [BUS]	(1) RW [BUS] (2) RW [BUS]	
2 ADDR	output	[11:0]	(1) IP1_INT [BUS] (2) IP2_INT [BUS]	(1) ADDR [BUS] (2) ADDR [BUS]	
3 WDATA	output	[31:0]	(1) IP1_INT [BUS] (2) IP2_INT [BUS]	(1) WDATA [BUS] (2) WDATA [BUS]	
4 IP1_SEL	output		IP1_INT [BUS]	IP1_INT [BUS]	
5 IP1_RDY	input		IP1_INT [BUS]	RDY [BUS]	
6 IP1_RDATA	input	[31:0]	IP1_INT [BUS]	RDY [BUS]	
7 IP2_SEL	output		IP2_INT [BUS]	RDY [BUS]	
8 IP2_RDY	input		IP2_INT [BUS]	RDY [BUS]	
9 IP2_RDATA	input	[31:0]	IP2_INT [BUS]	RDY [BUS]	

Assignment in Index/Slice:

Connection Setting

Port-to-Port | Port-to-Constant | Port-to-Wild | **Interface-to-Interface**

Select Target Interface

Target Instance:

Bus Interface	Bus Type	Logical Definition	Physical Ports
BUS_INT	BUS	(1) CLK (2) RW (3) ADDR (4) WDATA (5) SEL (6) RDY (7) RDATA	(1) IP_CLK (2) IP_RW (3) IP_ADDR (4) IP_WDATA (5) IP_SEL (6) IP_RDY (7) IP_RDATA

2) Select target interface

Select Source Interface

Bus Interface	Bus Type	Logical Definition	Physical Ports
IP1_INT	BUS	(1) CLK (7) RDATA	(1) CLK (7) IP1_RDATA
IP2_INT	BUS	(1) CLK (7) RDATA	(1) CLK (7) IP2_RDATA

3) Select source interface

4) Add physical ports within interface into connections list

Apply Ok Cancel

Interface-to-Wild

- Interface-to-Wildcard Connection
 - It is the same as port-to-wildcard flow except....
 - replacing ports with interfaces

Connect Port – Import by File

conn.list

Like “run.f” in design import, each line indicates the file (*.cia) for the port connection in each module scope

chip.u_p1_h1.cia
chip.u_p1_2.cia
chip.top.ccu.cia

<TARGET_INSTANCE>
chip.u_p1_h1

<MACRO_DEFINE>
HIER_0 chip.u_p1_m1

<NAME_RULE>
default

<MAP_TABLE>
p1 \$HIER_0.q1
p2 \$HIER_0.q2
p3 \$HIER_0.q3

<TARGET_INSTANCE>
chip.u_p1_2

<MACRO_DEFINE>
HIER_0 chip.u_p3_m1

<NAME_RULE>
default

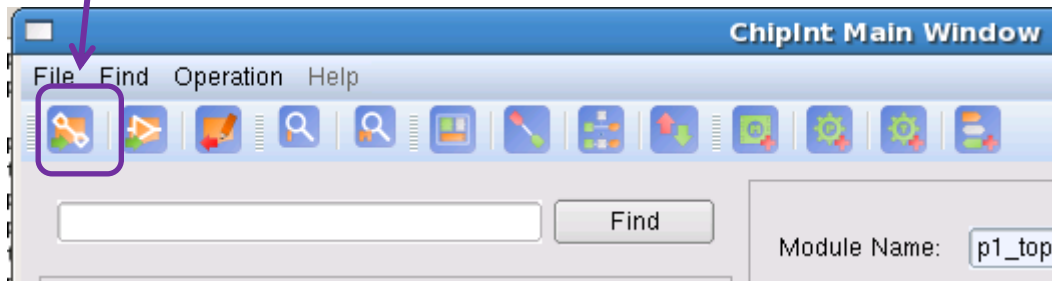
<MAP_TABLE>
p1 \$HIER_0.r1
p2 \$HIER_0.r2
p3 \$HIER_0.r3

<TARGET_INSTANCE>
chip.top.ccu

<MACRO_DEFINE>
HIER_0 chip.u_p4
HIER_1 chip.u_m1.u_m2

<NAME_RULE>
default

<MAP_TABLE>
p1 \$HIER_0.s1
p2 \$HIER_0.s2
p3 \$HIER_1.s3



Connect Port – Import by File

- How to get .cia or .list files?
 - Method1
 - By manual
 - Method2
 - ChipInt would export configuration files based on completed operations
 - Users can reuse these files for certain IP

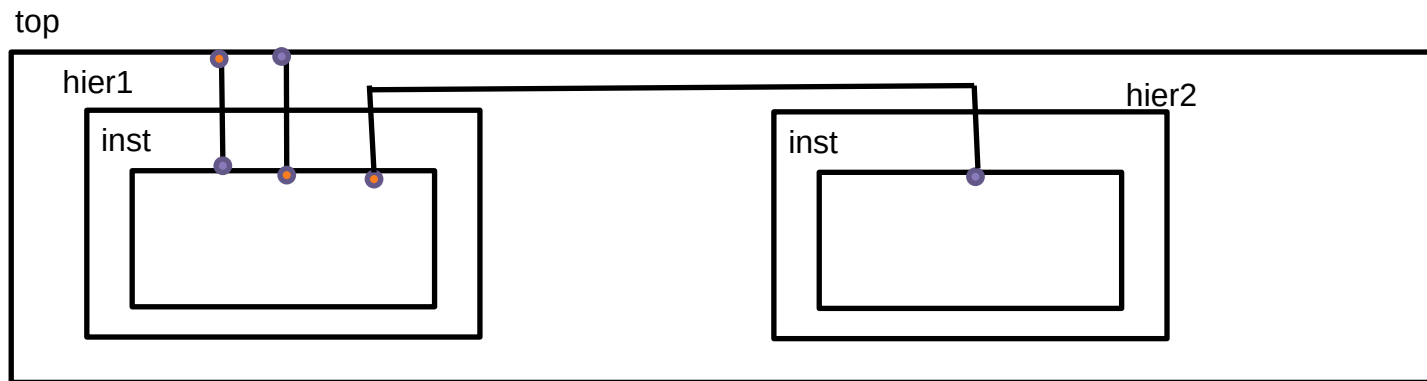
Outlines

- Overview
- Main Features
 - Integration operations
 - Module Instantiation
 - Port-to-Port Connection
 - Port-to-Hierarchy
 - Instance Insertion
 - Creation operations
 - Module Creation
 - Port Creation
 - Parameter Creation
 - Connection checking

Port-to-Hierarchy

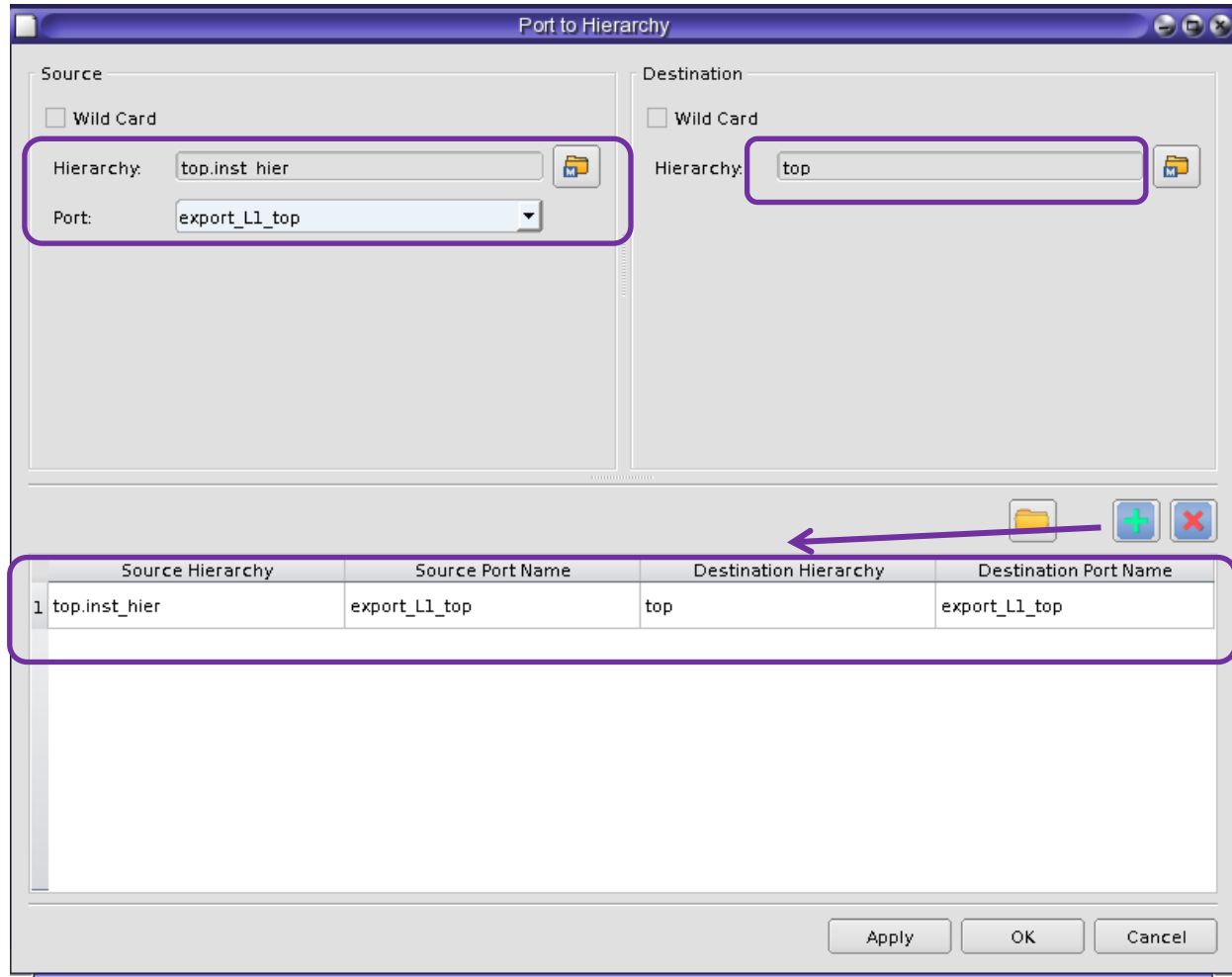
● Target hierarchy

● Source port



Port-to-Hierarchy

You can assign port-to-hierarchy one by one



Port-to-Hierarchy

You can assign multiple port-to-hierarchy at once with wild card.

Port to Hierarchy

Source

☒ Wild Card

Instance Name:

Port:

Destination

☒ Wild Card

Instance Name:

Add matched items into table

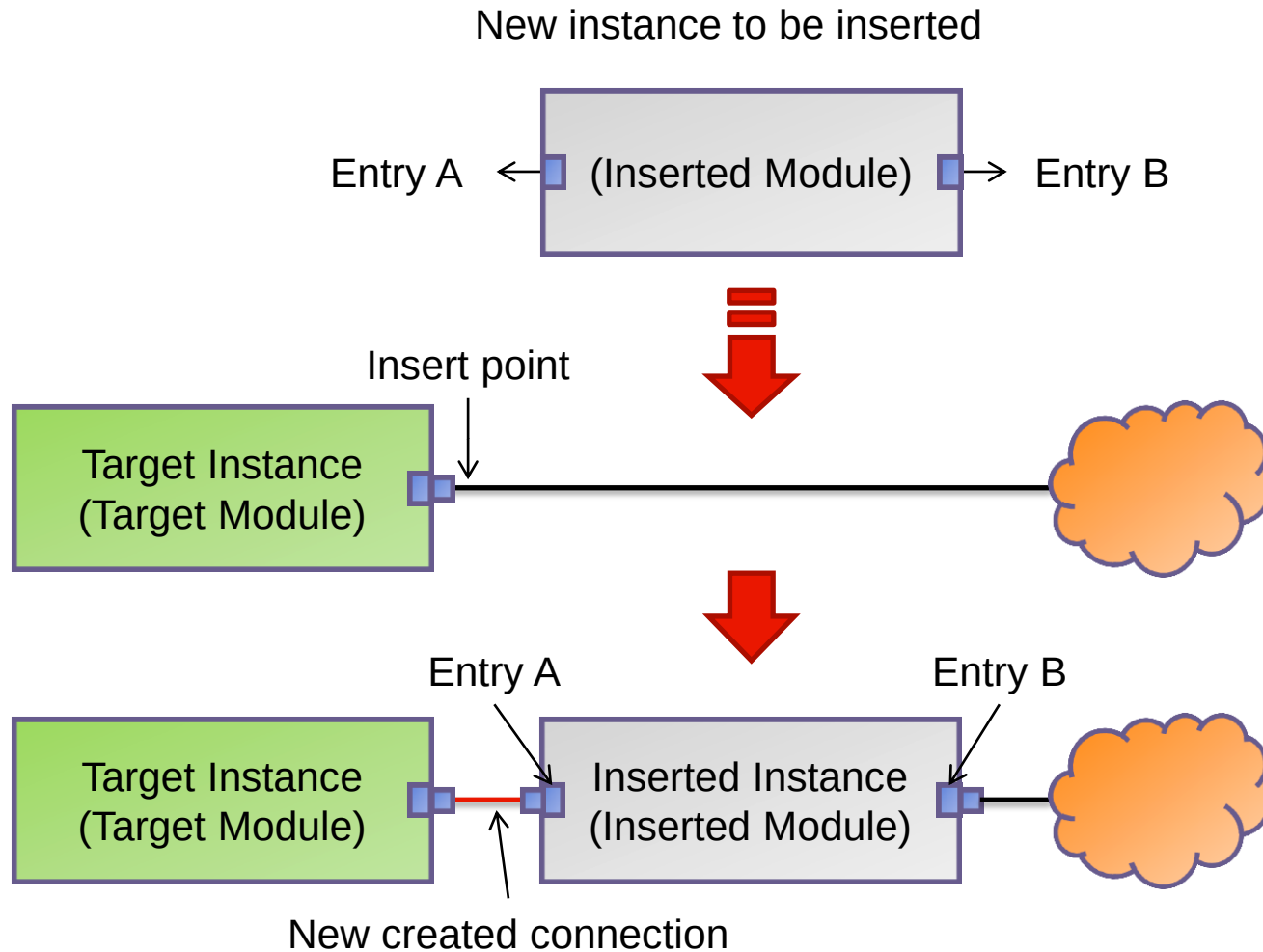
	Source Hierarchy	Source Port Name	Destination Hierarchy	Destination Port Name
1	top.u_chip.inst2	i1	top.u_chip.inst1	i1
2	top.u_chip.inst2	i2	top.u_chip.inst1	i2
3	top.u_chip.inst1	o1	top.u_chip.inst2	o1
4	top.u_chip.inst1	o2	top.u_chip.inst2	o2

Apply OK Cancel

Outlines

- Overview
- Main Features
 - Integration operations
 - Module Instantiation
 - Port-to-Port Connection
 - Port-to-Hierarchy
 - **Instance Insertion**
 - Creation operations
 - Module Creation
 - Port Creation
 - Parameter Creation
 - Connection checking

Instance Insertion



Instance Insertion – Under Scope

Insert an **instance** into an **instance port** under the specified **scope**

Insert Instance

Insert Point: Under Scope | Under Hierarchy | By Module

Scope: system

Instance: i_cpu

Port: clock

New Instance to Be Inserted:

Module: C_BUF

Entry A: out

Entry B: in

Instance Name: u_buf

☒ Smart Naming (<Prefix>_<Insert Point>_<Instance Name>_<Postfix>)

Prefix: Postfix:

	Scope	Instance	Port	Inserted Module	Inserted Instance	Entry A	Entry B
1	system	i_cpu	clock	C_BUF	u_buf	out	in

Apply OK Cancel

Diagram 1: Target Instance (Target Module) - Insert point - Original connected net

Diagram 2: New Instance to be inserted - Entry A - Entry B

Apply smart naming for all the inserted instance

Instance Insertion – Under Hierarchy

Insert an **instance** into an **instance port** of the specified instance **hierarchy**

Insert Instance

Insert Point

Under Scope Under Hierarchy By Module

Instance Hierarchy: system.i_cpu

Port: clock

New Instance to Be Inserted

Module: C_BUF

Entry A: out

Entry B: in

Instance Name: u_buf

☒ Smart Naming (<Prefix>_<Insert Point>_<Instance Name>_<Postfix>)

Prefix: Postfix:

	Scope	Instance	Port	Inserted Module	Inserted Instance	Entry A	Entry B
1	system	i_cpu	clock	C_BUF	u_buf	out	in

Apply OK Cancel

Instance Insertion – By Module

Insert an instance into **instance port(s)** of the instances that are instantiated by the specified **module**

Insert Instance

Insert Point

Under Scope Under Hierarchy **By Module**

Module: CPU

Port: clock

	Scope	Instance	Select
1	system	i_cpu	☐
2	system	i_cpu2	☐

New Instance to Be Inserted

Module: C_BUF

Entry A: out

Entry B: in

Instance Name: u_buf

☒ Smart Naming (<Prefix>_<Insert Point>_<Instance Name>_<Postfix>)

Prefix: Postfix:

	Scope	Instance	Port	Inserted Module	Inserted Instance	Entry A	Entry B
1	system	i_cpu	clock	C_BUF	u_buf	out	in

Apply OK Cancel

Insert Instance – Import by File

Do not modify first line

Insert Instance

Insert Point: _____ New Instance to Be Inserted: _____

By _____

Scope, Instance, Port, Ins. Module Name, Ins. Instance Name, Ins. Entry A, Ins. Entry B

system,i_cpu,clock,my_buf,u_clk_buf,out,in

system,i_cpu,clock,my_buf,u_clk_buf,out,in

system,i_cpu,clock,my_not,u_clk_not,out,in

system,i_cpu,clock,my_not,u_clk_not,out,in

system,i_cpu,clock,my_not,u_clk_not,out,in

☐ Smart Naming (<Prefix>_<Insert Point>_<Instance Name>_<Postfix>)

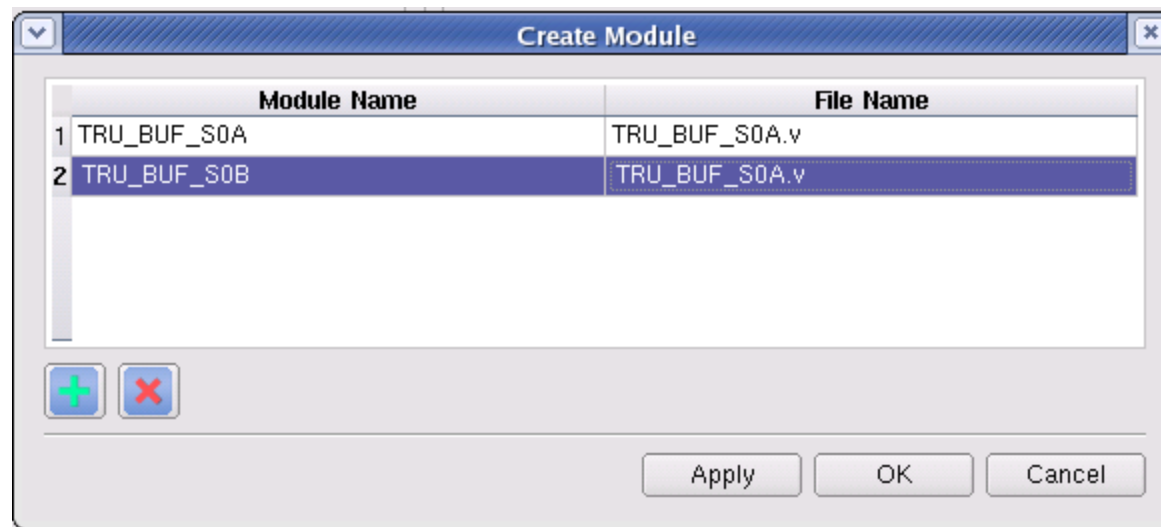
	Scope	Instance	Port	Ins. Module Name	Ins. Instance Name	Ins. Entry A	Ins. Entry B
1	system	i_cpu	clock	my_buf	u_clk_buf	out	in
2	system	i_cpu	clock	my_buf	u_clk_buf	out	in
3	system	i_cpu	clock	my_not	u_clk_not	out	in
4	system	i_cpu	clock	my_not	u_clk_not	out	in
5	system	i_cpu	clock	my_not	u_clk_not	out	in

Apply OK Cancel

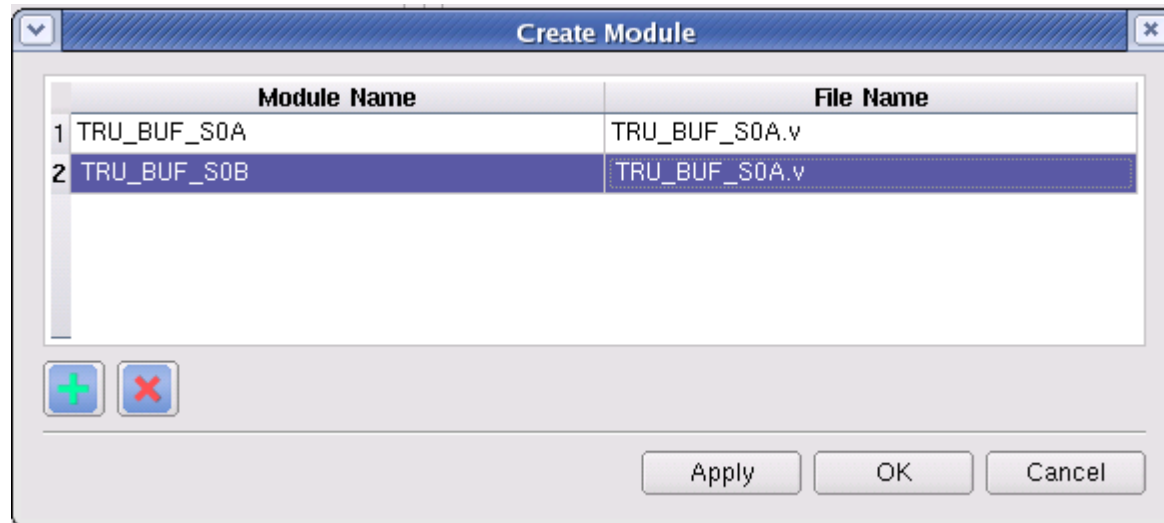
Outlines

- Overview
- Main Features
 - Integration operations
 - Module Instantiation
 - Port-to-Port Connection
 - Port-to-Hierarchy
 - Instance Insertion
 - Creation operations
 - Module Creation
 - Port Creation
 - Parameter Creation
 - Connection checking

Module Creation



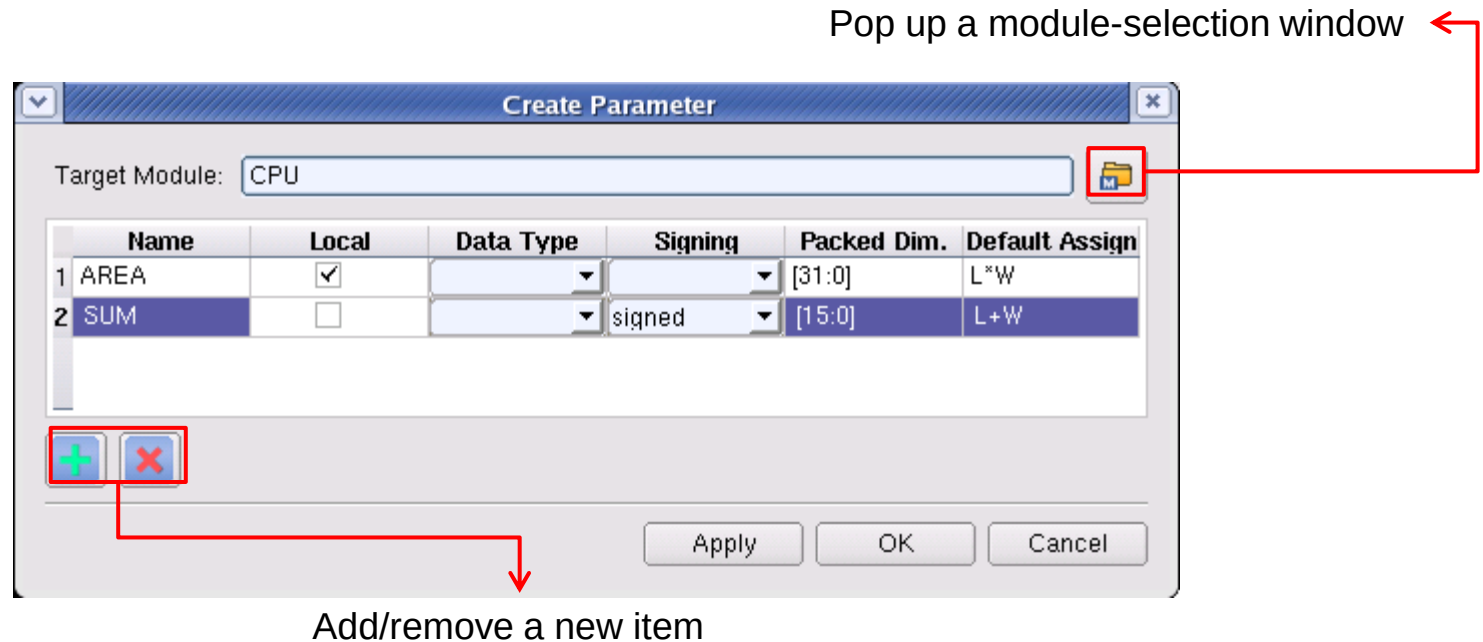
Example – Module Creation



```
// TRU_BUF_S0A.v  
module TRU_BUF_S0A;  
endmodule
```

```
// TRU_BUF_S0A.v  
module TRU_BUF_S0B;  
endmodule
```

Parameter Creation



Notes:

- When the target module is changed and the table is not empty, a warning dialog will be shown
 - Click Yes -> clean and update table content
 - Click No -> do nothing
- Data types contain unpacked dimensions are not allowed

Example – Parameter Creation

Target Module: CPU

	Name	Local	Data Type	Signing	Packed Dim.	Default Assign
1	AREA	☒			[31:0]	L*W
2	SUM	☐		signed	[15:0]	L+W

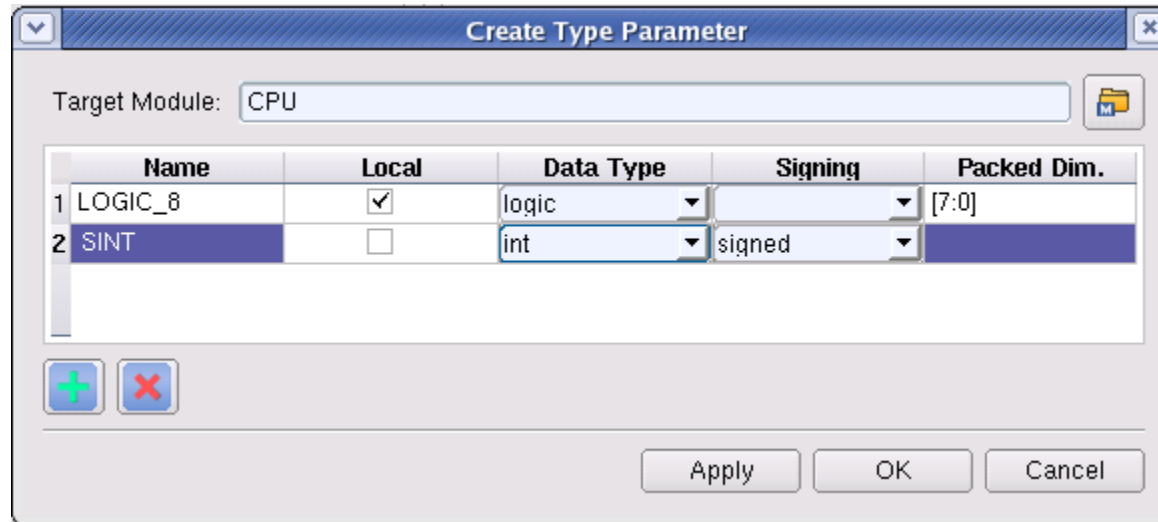
Apply OK Cancel

```
module CPU;  
parameter logic [15:0] L = 3;  
parameter logic [15:0] W = 4;  
endmodule
```

```
module CPU;  
parameter logic [15:0] L = 3;  
parameter logic [15:0] W = 4;  
localparam logic [31:0] AREA = L*W;  
parameter signed [15:0] SUM= L+W;  
endmodule
```

Parameter order will be handle by DM L0 automatically

Type Parameter Creation



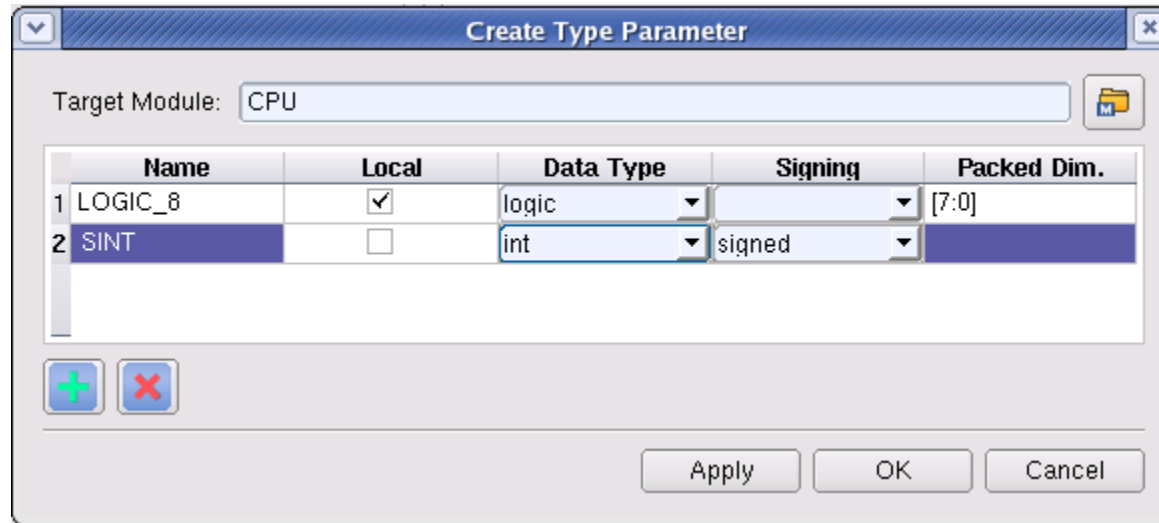
The image shows a 'Create Type Parameter' dialog box. At the top, the title bar reads 'Create Type Parameter'. Below the title bar, there is a 'Target Module:' label followed by a text field containing 'CPU'. To the right of the text field is a folder icon. Below this is a table with five columns: 'Name', 'Local', 'Data Type', 'Signing', and 'Packed Dim.'. The table contains two rows. Row 1 has 'LOGIC_8' in the 'Name' column, a checked checkbox in the 'Local' column, 'logic' in the 'Data Type' column, an empty dropdown in the 'Signing' column, and '[7:0]' in the 'Packed Dim.' column. Row 2 has 'SINT' in the 'Name' column, an unchecked checkbox in the 'Local' column, 'int' in the 'Data Type' column, 'signed' in the 'Signing' column, and an empty field in the 'Packed Dim.' column. Below the table are two buttons: a green plus sign and a red minus sign. At the bottom right are three buttons: 'Apply', 'OK', and 'Cancel'.

Target Module: CPU

	Name	Local	Data Type	Signing	Packed Dim.
1	LOGIC_8	☒	logic		[7:0]
2	SINT	☐	int	signed	

Apply OK Cancel

Example – Type Parameter Creation



The dialog box titled "Create Type Parameter" has a "Target Module:" field set to "CPU". Below this is a table with five columns: "Name", "Local", "Data Type", "Signing", and "Packed Dim.". The table contains two entries: "LOGIC_8" (Local: checked, Data Type: logic, Signing: (empty), Packed Dim.: [7:0]) and "SINT" (Local: unchecked, Data Type: int, Signing: signed, Packed Dim.: (empty)). At the bottom left are two buttons with a green plus and a red minus. At the bottom right are "Apply", "OK", and "Cancel" buttons.

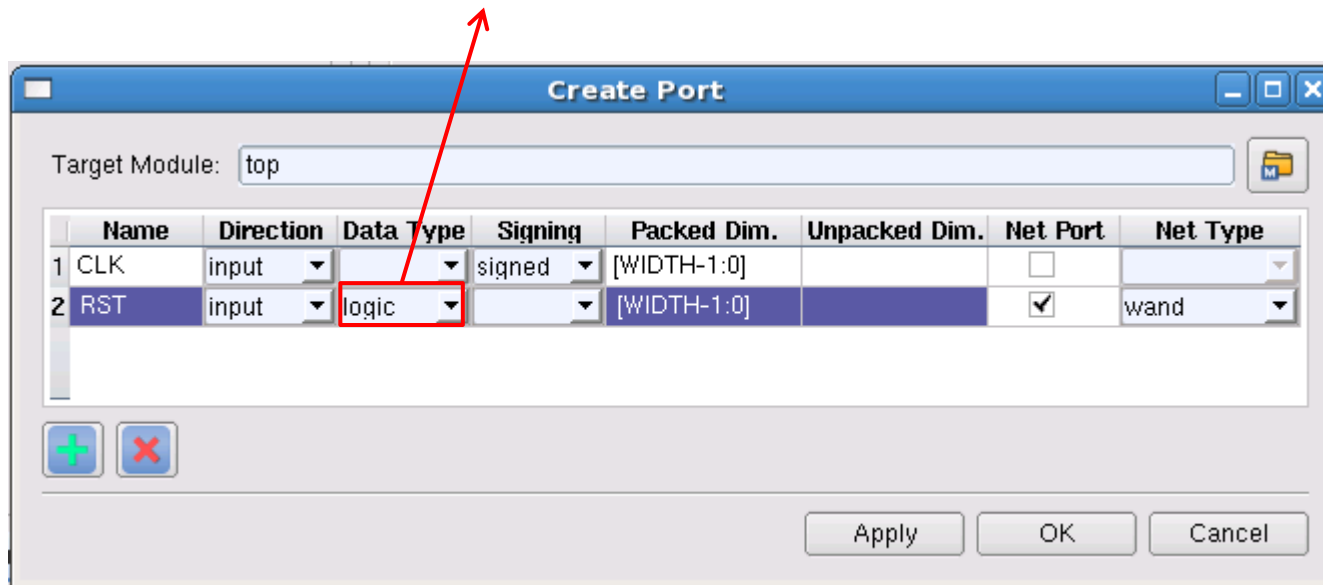
	Name	Local	Data Type	Signing	Packed Dim.
1	LOGIC_8	☒	logic		[7:0]
2	SINT	☐	int	signed	

```
module cpu;  
endmodule
```

```
module CPU;  
localparam type LOGIC_8 = logic [7:0];  
parameter type SINT= int signed;  
endmodule
```

Port Creation

When net port is selected, only 4-state integral data types will be shown, all the UDP will be filtered.



Example – Port Creation

Target Module: top

	Name	Direction	Data Type	Signing	Packed Dim.	Unpacked Dim.	Net Port	Net Type
1	CLK	input		signed	[WIDTH-1:0]		☐	
2	RST	input	logic		[WIDTH-1:0]		☒	wand

+ x

Apply OK Cancel

```
module CPU;  
parameter [2:0] WIDTH = 3;  
endmodule
```



```
module CPU(CLK, RST);  
parameter [2:0] WIDTH = 3;  
input reg signed [WIDTH-1:0] CLK;  
input [WIDTH-1:0] RST;  
wand logic [WIDTH-1:0] RST;  
endmodule
```

Outlines

- Overview
- Main Features
 - Integration operations
 - Module Instantiation
 - Port-to-Port Connection
 - Port-to-Hierarchy
 - Instance Insertion
 - Creation operations
 - Module Creation
 - Port Creation
 - Parameter Creation
 - Connection checking

Connection Checking View

When the instance is selected, the port tree will display all the port of this instance.

ChipInt Main Window

File Find Operation

Module Name: ALUB

Instance Name: tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB

Port Tree

Name	Type
RESET	wire
CLOCK1	wire
CLOCK2	wire
CLOCK3	wire
CLOCK4	wire
OpBusMode[2:0]	wire
AluMode[2:0]	wire
AluMode[2]	wire
AluMode[1]	wire
AluMode[0]	wire
CarryMode	wire
ResultBusMode[2:0]	wire
DataIn[7:0]	wire
PC[7:0]	wire
ACC[7:0]	wire

Connection Port List

	Name	Hier Name
0	HD	AluMode
1	LL	select
2	LS	case statement
3	LS	event control
4	HS	assignment
5	HS	assignment

Instance_Tree Decl_Tree

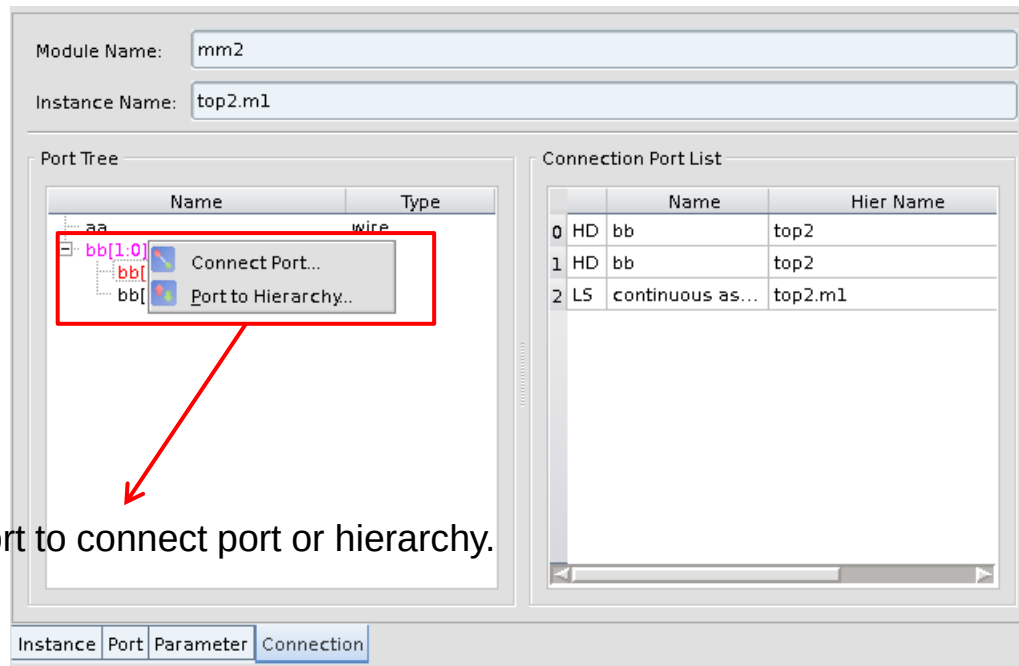
Message Output Files Statistics

Instance Port Parameter Connection

When the port is selected, the connection port list will display all the connection port and statement of this port.

Floating Port Checking

- Check any bit of port is connected any logic cell, expression or other ports.
 - Black: fully connected.
 - Purple: partial connected.
 - Red: floating.



Right click on the port to connect port or hierarchy.

Port Connection Checking

Right click on the connected port which you want to know how it connect to the target port.

The screenshot shows the Chipint main window with the following components:

- Module Name:** ALUB
- Instance Name:** tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB
- Port Tree:** A list of ports and their types. The ports are: RESET (wire), CLOCK1 (wire), CLOCK2 (wire), CLOCK3 (wire), CLOCK4 (wire), OpBusMode[2:0] (wire), AluMode[2:0] (wire), CarryMode (wire), ResultBusMode[2:0] (wire), DataIn[7:0] (wire), PC[7:0] (wire), ACC[7:0] (wire), IXR[7:0] (wire), TR[7:0] (wire), AluOut[7:0] (wire), AluBuf[7:0] (reg), DataOut[7:0] (reg), CFout (wire), and ZFout (wire).
- Connection Port List:** A table showing connections. The table has columns: Name, Hier Name, and a button 'Show path'. The connections are: 0 HD CLOCK4 tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4, 1 HD CLOCK4 tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4, 2 LS event_control tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4, 3 HS assignment tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4, 4 HS assignment tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4, 5 HS assignment tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4, and 6 HS assignment tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4. The 'Show path' button is highlighted with a red box.
- Message Console:** A text area showing the output of the 'Show path' button. The output is: 'The path of port "tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4" to stmt "assignment": [0] tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.CLOCK4, [1] tb_CPUsystem.i_CPUsystem.i_CPU.CLOCK4, [2] tb_CPUsystem.i_CPUsystem.CLOCK4, [3] CLOCK4 = 0;'. The message console is also highlighted with a red box.

The message console will show the path from target port to connected port.

Port Connection Checking

- There are 6 tags to indicate each connection type.
 - HD: High-connection Driver
 - LD: Low-connection Driver
 - HL: High-connection Load
 - LL: Low-connection Load
 - HS: High-connection Statement
 - LS: Low-connection Statement

The screenshot shows the Synopsys Port Connection Checking tool interface. At the top, the 'Module Name' is 'mm2' and the 'Instance Name' is 'top2.m1'. Below this, there are two main panels: 'Port Tree' and 'Connection Port List'.

Port Tree: A tree view showing the hierarchy of ports. The root is 'aa', which has a child 'bb[1:0]'. 'bb[1:0]' has two children: 'bb[1]' and 'bb[0]'. The 'Type' column indicates that 'aa' is a 'wire', 'bb[1:0]' is a 'wire', 'bb[1]' is a 'wire', and 'bb[0]' is a 'wire'.

Connection Port List: A table showing the connection details for the selected port 'bb[1:0]'. The table has columns for 'Name' and 'Hier Name'.

	Name	Hier Name
0 HD	bb	top2
1 HD	High-connection Driver	
2 LS	continuous as...	top2.m1

At the bottom of the interface, there are tabs for 'Instance', 'Port', 'Parameter', and 'Connection'. The 'Connection' tab is currently selected.